

VisionEngine

Ver la interfaz como un usuario: visión por computadora más visión LLM para análisis y navegación.

Resumen

VisionEngine es un conjunto de herramientas Go desacoplado que combina la visión por computadora clásica con la visión basada en LLM para analizar interfaces de usuario, detectar elementos de la UI e identificar problemas visuales, además de construir grafos de navegación de las transiciones entre pantallas de una aplicación, con backends de visión multi-proveedor conectables y OpenCV protegido tras una etiqueta de compilación.

Descripción breve

Un módulo Go reutilizable para el análisis de interfaces y la construcción de grafos de navegación. Ofrece una capa de analizador (elementos de UI, diferencias entre pantallas, problemas visuales), un grafo de navegación con búsqueda de rutas mediante BFS y exportación en DOT/JSON/Mermaid, y adaptadores de visión LLM para GPT-4o, Claude, Gemini, Qwen-VL y otros.

Descripción detallada

La mayoría de las automatizaciones de pruebas de interfaces son, en la práctica, ciegas. Recurren a árboles de accesibilidad y selectores DOM —la idea que tiene una máquina de una interfaz— y pasan por alto todo lo que experimenta un ser humano: si un botón se renderiza de forma visible, si el diseño se rompió o si la pantalla a la que llegó es la esperada. VisionEngine cierra esa brecha al dotar a la automatización de una percepción genuina, la capacidad de observar una interfaz y razonar sobre ella como lo haría una persona. Está organizado en cuatro capas cooperativas que parten de píxeles crudos hasta alcanzar una comprensión integral de la aplicación.

El **Analizador** define el contrato estable —interfaces (Analyzer, VideoProcessor) y tipos de valor (UIElement, ScreenAnalysis, ScreenDiff, Rect, Size, TextRegion, VisualIssue, ScreenIdentity, Action, KeyFrame) con una implementación de referencia StubAnalyzer— para que los consumidores puedan detectar elementos, comparar pantallas e identificar problemas visuales bajo un contrato que no cambiará bajo sus pies.

El **Grafo de Navegación** eleva la perspectiva de una sola pantalla a toda la aplicación, modelándola como un grafo dirigido de transiciones entre pantallas con búsqueda de rutas mediante BFS y tres backends de exportación (DOT, JSON, Mermaid). Así, la automatización no solo puede “ver” una pantalla, sino también planificar una ruta hacia cualquier otra, y se incluye con suites de pruebas de estrés, automatización, integración y seguridad para validarlo.

La capa de **Visión LLM** añade razonamiento multimodal moderno: una interfaz VisionProvider con adaptadores para OpenAI (GPT-4o), Anthropic (Claude), Gemini, Qwen-VL, Kimi, StepGUI, Astica y Ollama, compuesta mediante una FallbackChain para que un proveedor fallido, limitado por tasa o débil degrade de forma elegante al siguiente en lugar de arrastrar consigo la ejecución.

Una capa de **Configuración** gestiona la carga y validación de variables de entorno, con todos los mensajes de error orientados al usuario canalizados a través del i18n.Translator.

La decisión que hace que todo esto sea realmente adoptable es que la dependencia nativa pesada es opcional. Los enlaces de OpenCV están protegidos tras una etiqueta de compilación -tags vision, y la compilación predeterminada incluye stubs, de modo que el módulo completo se compila, prueba y ejecuta en cualquier host Go 1.25+ sin necesidad de una cadena de herramientas OpenCV, incorporando el stack nativo solo cuando el consumidor lo solicita explícitamente. Esto permite que VisionEngine se integre en un ejecutor de CI estándar sin requerir una imagen personalizada. Totalmente desacoplado según lo establecido en la constitución (CONST-051(B)), es incorporado por los consumidores —en particular HelixQA— como un submódulo de código base equivalente, dotando a las pruebas de interfaces basadas en evidencia de un par de ojos reales.

Por qué lo creamos

La automatización de pruebas de interfaz de usuario que depende únicamente de árboles de accesibilidad o selectores pasa por alto lo que el usuario realmente ve. VisionEngine incorpora comprensión visual real —detección de elementos, comparación de pantallas y razonamiento basado en visión de LLM—, además

de un mapa navegable de las pantallas de la aplicación, para que la automatización pueda tanto percibir como desplazarse por una interfaz de usuario.

Por qué es un cambio radical

Combina dos enfoques normalmente incompatibles —la visión por computadora clásica, rápida y determinista, y la visión flexible y semántica de LLM— bajo una misma interfaz con una cadena de respaldo, de modo que el usuario obtiene la precisión de uno y el razonamiento del otro sin tener que elegir. Además, al mantener OpenCV como opción estrictamente opcional, elimina el coste habitual de ese poder: cualquier proyecto de Go puede adquirir percepción real de la interfaz sin arrastrar una cadena de herramientas de visión nativa a su compilación.

Qué hay de innovador

- Percepción dual: visión por computadora clásica (OpenCV/GoCV) más visión de LLM con múltiples proveedores y cadena de respaldo.
- Grafo de navegación con búsqueda en anchura (BFS) y exportación a DOT/JSON/Mermaid.
- OpenCV con etiquetas de compilación para que el módulo siga siendo compilable y testeable sin dependencias nativas.
- Submódulo completamente desacoplado, con costuras de internacionalización y código base unificado (usado por HelixQA).

Desafíos y soluciones

- **Fricción por dependencias nativas pesadas:** resuelto con el control mediante `-tags vision` y stubs predeterminados para que CI/hosts sin OpenCV sigan compilando y probando.
- **Falta de fiabilidad en los proveedores de visión:** resuelto con una interfaz `VisionProvider` y un compositor `FallbackChain`.
- **Mapeo de flujos de aplicaciones complejos:** resuelto con un grafo de navegación dirigido, búsqueda en anchura (BFS) y exportación en múltiples formatos.
- **Acoplamiento:** resuelto mediante el desacoplamiento `CONST-051(B)` y la capa de traducción `i18n`.

Pila tecnológica (por qué y cómo)

- **Go (1.25+)** — núcleo del módulo y sus cuatro capas.

- **GoCV / OpenCV** — visión por computadora clásica, controlada por etiquetas de compilación.
- **Proveedores de visión de LLM (GPT-4o, Claude, Gemini, Qwen-VL, Kimi, StepGUI, Astica, Ollama)** — razonamiento multimodal de interfaces mediante adaptadores.
- **Algoritmos de grafos (BFS)** — búsqueda de rutas de navegación.
- **Exportadores DOT / JSON / Mermaid** — visualización del grafo de navegación.
- **Traductor i18n** — cadenas de texto para el usuario desacopladas.